

# Next-POI Recommendation

*A TLMR-faithful baseline on Foursquare NYC*

*Architecture, training procedure & results*

PyTorch · PyTorch Geometric · Google Colab T4

Repository: [github.com/6ym6n/PFE\\_IMPLEMENTATION](https://github.com/6ym6n/PFE_IMPLEMENTATION)

Final report — May 2026

# Executive summary

This report documents an end-to-end implementation of a next-POI (Point-of-Interest) recommender on the Foursquare NYC check-in benchmark. The architecture is a two-layer GCN over a hybrid POI graph (co-visit  $\cup$  geographic kNN) feeding a GRU sequence model concatenated with a user embedding, followed by an MLP scoring head over the full POI vocabulary. The implementation faithfully follows the TLMR (Triple-Layered Modular Recommender) spec used as the project blueprint, with no "silent improvements."

**Result.** On the standard NYC test split (per-user chronological 70/10/20), the trained model reaches **HR@1 = 0.187**, **HR@5 = 0.476**, **HR@10 = 0.588**, **NDCG@10 = 0.375**, and **MRR = 0.316**. HR@1 lands at the top of the LSTM/STGCN tier reported in the LLM4POI benchmark table — exactly the band targeted for an honest baseline. No leakage symptoms (HR@1 > 0.20 with this architecture would be suspicious).

The remainder of the report walks through the problem formulation, the architecture stage by stage with diagrams, the training procedure, the ranking metrics, and a discussion of the results in context.

## 1. Problem formulation

Let  $U$  be the set of users and  $V$  the vocabulary of POIs. Each user  $u \in U$  produces a chronological sequence of check-ins; we segment these into **sessions** by splitting at temporal gaps larger than 24 hours (Foursquare convention). A session of length  $L$  is an ordered tuple  $S = (p_1, \dots, p_L)$  with associated context features  $\Delta d_t$  (haversine distance from  $p_{t-1}$  to  $p_t$  in km) and  $\Delta t_t$  (time gap from  $p_{t-1}$  to  $p_t$  in hours).

The **next-POI prediction** task is: given a prefix  $(p_1, \dots, p_T)$  from a session and the user identity  $u$ , predict a probability distribution over  $V$  for the next POI  $p_{T+1}$ . Quality is measured by ranking metrics (HR@k, NDCG@k, MRR) computed over the entire POI vocabulary — no negative sampling, no candidate restriction.

$$P(p_{T+1} = v \mid S, u) = \text{softmax}(\hat{y})_v, \text{ where } \hat{y} = f_\theta(S, u; G) \in \mathbb{R}^{|V|}$$

## 2. Architecture overview

The model is a **three-stage pipeline**: (1) feature extraction lifts each input into a vector representation, (2) a sequence model integrates the prefix into a single context vector, and (3) a scoring head produces logits over all POIs.

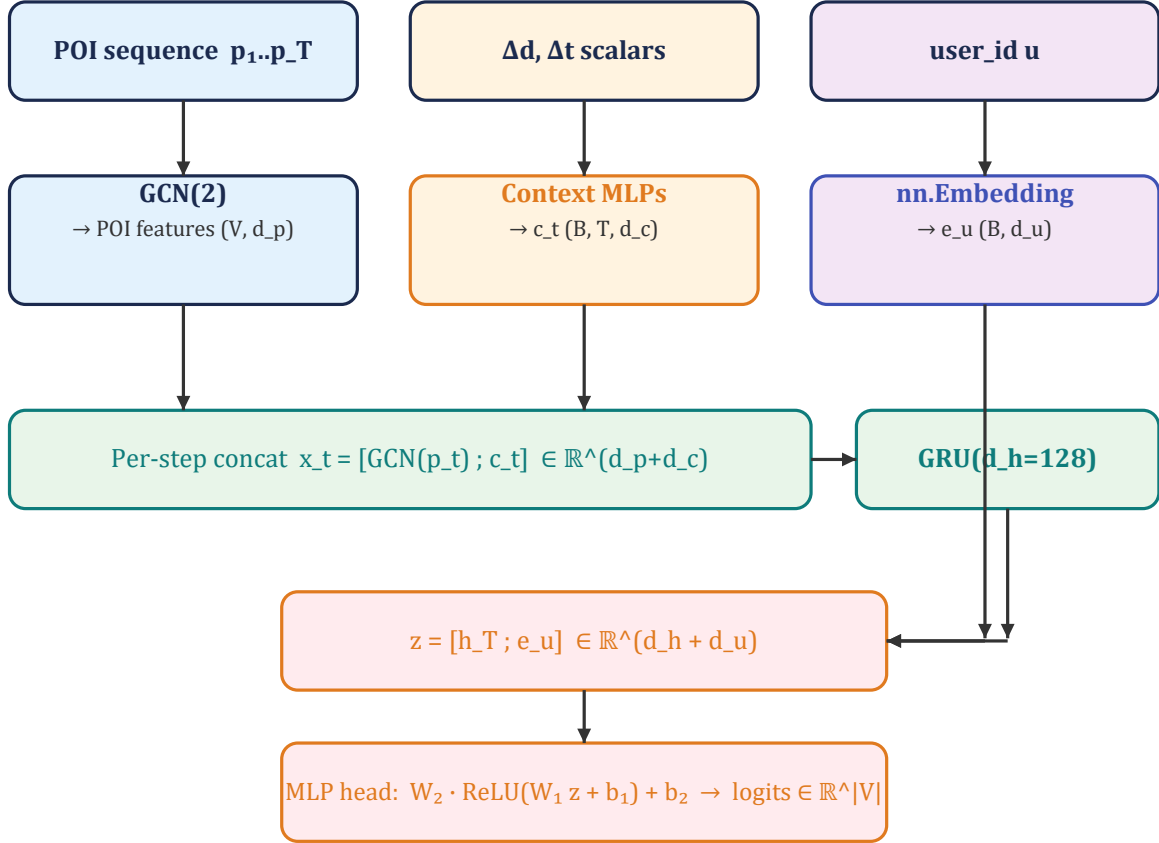


Figure 1 — End-to-end forward pipeline. Inputs (top): the POI prefix, per-step context scalars, and the user id. Each is encoded independently (middle row), POI features and context are concatenated per step, the sequence is consumed by a GRU, the final hidden state is concatenated with the user embedding, and an MLP head produces logits over the entire vocabulary.

### 3. Stage 1 — feature extraction

#### 3.1 Hybrid POI graph

POIs are not isolated tokens — they sit in geographic space and are linked by user behavior. The model exploits both signals by building a single hybrid graph  $G = (V, E_{cov} \cup E_{geo})$ :

- **Co-visit edges ( $E_{cov}$ ).** For every pair  $(p_i, p_j)$ , count how many distinct users transitioned  $p_i \rightarrow p_j$  as consecutive check-ins within a session, in **training data only**. Add the undirected edge if the count  $\geq \tau_{cov} = 3$ . Building this from train+val+test would be subtle leakage.
- **Geographic kNN edges ( $E_{geo}$ ).** Each POI is connected to its  $k = 10$  nearest neighbors by haversine distance, computed with a BallTree. This captures spatial proximity even for POIs with sparse co-visit history.

The two edge sets are unioned (de-duplicated) and symmetrized into a PyG edge\_index tensor of shape  $(2, 2|E|)$ . On NYC the resulting graph has 5,284 co-visit + 31,790 kNN edges, union 34,898, with degree min/median/max = 10/13/79 and **no isolated nodes** — every POI receives graph signal.

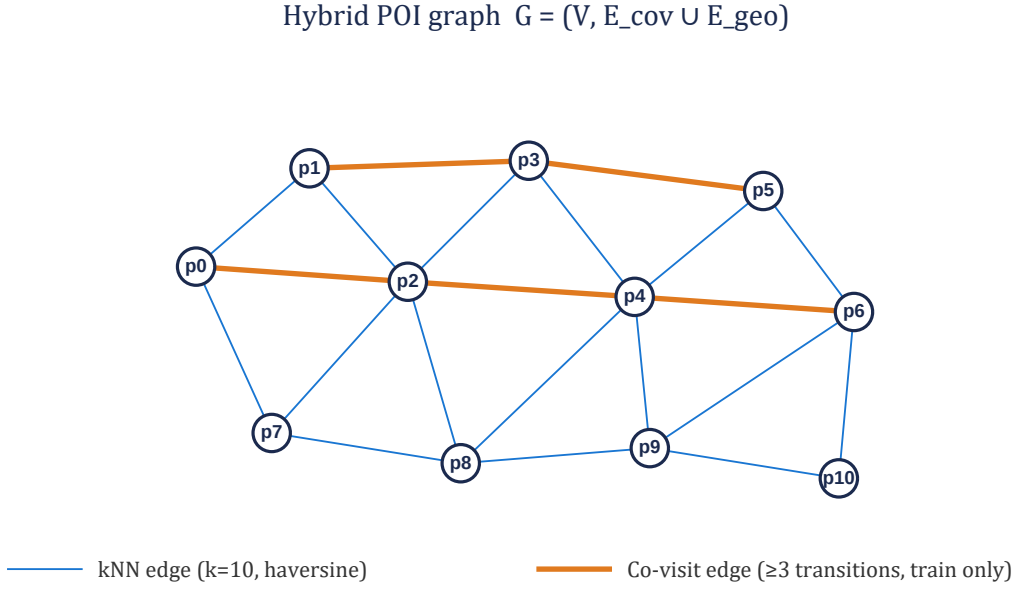


Figure 2 — Hybrid POI graph (illustrative subset). Thin blue edges are geographic kNN; thick orange edges are training-derived co-visit transitions. Co-visit edges connect POIs that users actually traveled between, even if they are not geographically close.

### 3.2 GCN encoder

A learnable embedding table  $E^{(0)} \in \mathbb{R}^{|V| \times d_p}$  ( $d_p = 128$ , Xavier init) holds initial POI embeddings. Two GCN layers propagate them through the hybrid graph using the standard symmetric normalization:

$$E^{(\ell+1)} = \sigma(D^{-1/2} \tilde{A} D^{-1/2} E^{(\ell)} W^{(\ell)}), \ell = 0, 1$$

where  $\tilde{A} = A + I$  adds self-loops and  $D$  is the corresponding degree matrix. ReLU + dropout( $p=0.2$ ) is applied between the two layers. The output  $h_p^{\text{GCN}} = E^{(2)} \in \mathbb{R}^{|V| \times d_p}$  is the matrix of POI features used downstream — the sequence model gathers rows from this matrix by POI index. Two layers is the standard depth: more leads to over-smoothing (POI features collapse toward each other).

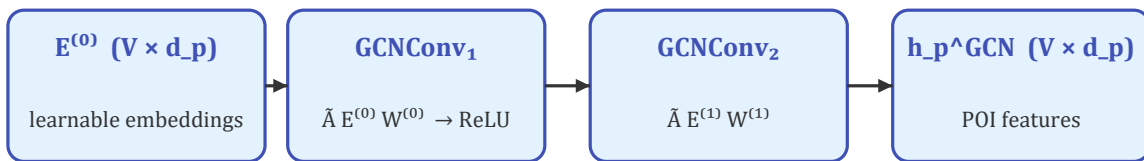


Figure 3 — Two-layer GCN over the hybrid graph. The full vocabulary is propagated once per forward pass (not per example), then indexed by the input POI ids.

### 3.3 Context encoder

Per-step context is the haversine distance  $\Delta d_t$  and the time gap  $\Delta t_t$  from the previous POI in the session. Two parallel MLPs lift each scalar into  $\mathbb{R}^{16}$ , then the halves are concatenated:

$$c_t = [ \text{MLP}_d(\Delta d_t) ; \text{MLP}_t(\Delta t_t) ] \in \mathbb{R}^{d_c=32}$$

Each MLP is  $\text{Linear}(1 \rightarrow 16) \rightarrow \text{ReLU} \rightarrow \text{Linear}(16 \rightarrow 16)$ . At session boundaries the previous POI is undefined, so  $\Delta d_1 = \Delta t_1 = 0$  by convention.  $\Delta d$  is clipped to  $[0, 100]$  km and  $\Delta t$  to  $[0, 24]$  hours, which prevents extreme outliers (e.g. a typo in a timestamp) from destabilizing the MLPs.

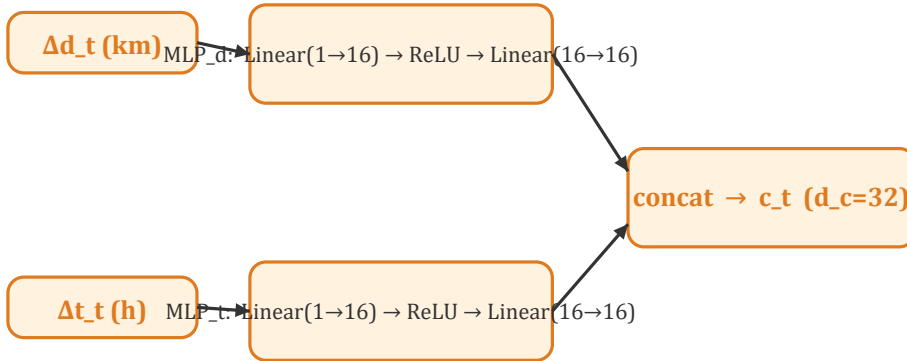


Figure 4 — Context encoder. Two independent MLPs let the model learn separate non-linear mappings for spatial and temporal gaps before they are combined.

### 3.4 User embedding

Each user has a learnable embedding  $e_u \in \mathbb{R}^{(d_u=64)}$  stored in an `nn.Embedding(|U|, 64)` table (Xavier init). This captures stable, user-specific preferences (e.g. a user who systematically prefers cafés to bars) that are not present in the session prefix itself.

## 4. Stage 2 — sequence model

### 4.1 GRU encoder

Per-step input is the concatenation of POI feature and context:  $x_t = [h_{\{p_t\}^{GCN}}; c_t] \in \mathbb{R}^{(d_p + d_c)} = \mathbb{R}^{160}$ . Sessions in a minibatch have variable length, so they are right-padded to the batch maximum and consumed by the GRU via `pack_padded_sequence` (lengths must live on CPU for PyTorch's packing). The final hidden state  $h_T \in \mathbb{R}^{(d_h=128)}$  summarizes the entire prefix.

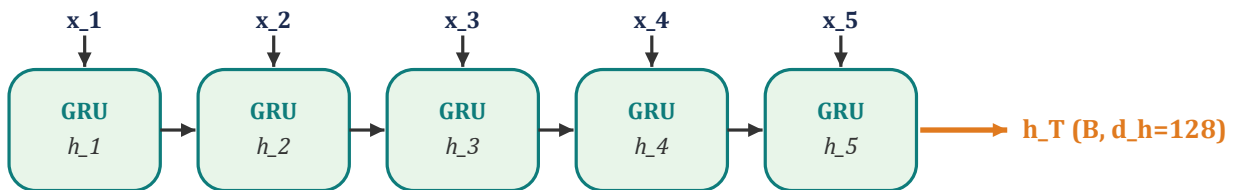


Figure 5 — GRU unrolled over a session prefix. Only the final hidden state  $h_T$  is propagated downstream; intermediate states are discarded.

### 4.2 Scoring head

The user embedding is concatenated with the GRU summary,  $z = [h_T; e_u] \in \mathbb{R}^{(d_h + d_u)} = \mathbb{R}^{192}$ , and fed to a two-layer MLP with hidden dim 256:

$$\hat{y} = W_2 \cdot \text{ReLU}(W_1 z + b_1) + b_2 \in \mathbb{R}^{|V|}$$

Output dimension is the full POI vocabulary (5,135 on NYC after filtering) so this is the parameter-heaviest layer. Dropout( $p=0.2$ ) is applied after the ReLU. The softmax is implicit in the cross-entropy loss; for ranking we use the raw logits.

## 5. Training procedure

Each session of length  $L$  is converted into  $L-1$  supervised examples by taking every prefix (length  $1..L-1$ )  $\rightarrow$  next-POI pair. Histories are right-padded per batch and consumed by the model in a single pass. The loss is plain cross-entropy over the  $|V|$ -way logits:

$$L = - (1/N) \cdot \sum_i \log P(y_i | S_i, u_i)$$

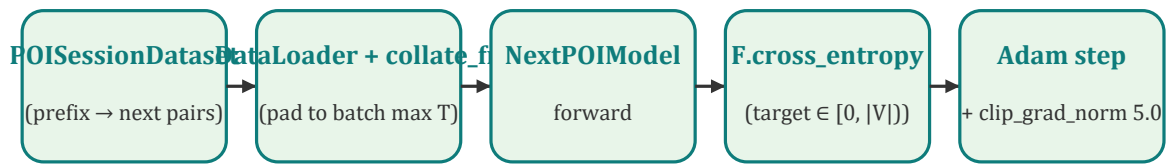


Figure 6 — Training pipeline. The GCN runs once per forward pass on the entire vocabulary (not per example), then per-step POI features are gathered by index — this is critical for throughput.

### 5.1 Hyperparameters (locked from spec)

| Hyperparameter                  | Value                                                        |
|---------------------------------|--------------------------------------------------------------|
| POI embedding dim $d_p$         | 128                                                          |
| User embedding dim $d_u$        | 64                                                           |
| Context dim $d_c$               | 32 (16 + 16)                                                 |
| GRU hidden dim $d_h$            | 128                                                          |
| MLP head hidden dim             | 256                                                          |
| GCN layers                      | 2                                                            |
| Dropout                         | 0.2                                                          |
| Optimizer                       | Adam (lr=1e-3, weight_decay=1e-5)                            |
| Batch size                      | 64                                                           |
| Max epochs / patience           | 50 / 8 (early stop on val HR@10)                             |
| Gradient clip (L2 norm)         | 5.0                                                          |
| Co-visit threshold $\tau_{cov}$ | 3 (training data only)                                       |
| Geographic kNN k                | 10 (haversine)                                               |
| Filter                          | $\geq 10$ check-ins/user, $\geq 10$ visitors/POI (iterative) |
| Session split gap               | 24 h                                                         |

Train / val / test

70 / 10 / 20 chronological per user

Random seed

42

## 6. Evaluation metrics

Given the logits  $\hat{y} \in \mathbb{R}^{|V|}$  and the true next POI  $y$ , the **rank** of the target is the number of POIs scored strictly higher than the target, plus one:

$$\text{rank}(y) = |\{v \in V : \hat{y}_v > \hat{y}_y\}| + 1$$

All three metrics derive from this rank, averaged over every test example:

### 6.1 Hit Rate at k (HR@k)

$$\text{HR@k} = (1/N) \cdot \sum_i [\text{rank}(y_i) \leq k]$$

Fraction of examples where the true next POI appears in the top-k recommendations. HR@1 is the strictest — it equals the exact-match accuracy.

### 6.2 Normalized Discounted Cumulative Gain (NDCG@k)

$$\text{NDCG@k} = (1/N) \cdot \sum_i [\text{rank}(y_i) \leq k] / \log_2(\text{rank}(y_i) + 1)$$

Like HR@k, but rewards higher ranks more — a target at rank 1 contributes 1, at rank 2 contributes  $1/\log_2(3) \approx 0.63$ , and so on. Standard ranking metric in IR and recommender systems.

### 6.3 Mean Reciprocal Rank (MRR)

$$\text{MRR} = (1/N) \cdot \sum_i 1 / \text{rank}(y_i)$$

Average of the reciprocal of the rank — captures the full ranking quality without a cutoff. A model that ranks the target second on every example would score MRR = 0.5; one that always ranks it first would score 1.0.

All metrics are computed over the **full POI vocabulary** (no negative sampling, no candidate restriction). This matches the GETNext / STHGCN evaluation protocol used by the LLM4POI benchmark we compare against — a critical detail since candidate-restricted evaluation can inflate metrics by 2–3×.

## 7. Results on NYC

### 7.1 Data pipeline sanity

Each preprocessing stage was instrumented; here are the actual numbers from the run:

| Stage                                        | Check-ins | Users           | POIs           |
|----------------------------------------------|-----------|-----------------|----------------|
| Raw download                                 | 227,428   | 1,083           | 38,333         |
| After iterative filter ( $\geq 10/\geq 10$ ) | 147,938   | 1,083           | 5,135          |
| After sessionization ( $\geq 2$ per session) | —         | 23,867 sessions | avg length 5.2 |
| Train split (70%)                            | 91,148    | 16,232 sessions | —              |
| Val split (10%)                              | 8,825     | 1,909 sessions  | —              |

|                  |        |                |   |
|------------------|--------|----------------|---|
| Test split (20%) | 24,734 | 5,726 sessions | — |
|------------------|--------|----------------|---|

Hybrid graph: 5,284 co-visit edges + 31,790 kNN edges, union 34,898 unique edges. Degree distribution min=10, median=13, mean=13.6, max=79. Zero isolated POIs (every node receives graph signal).

Materialized examples after generating prefix→target pairs: **74,916 train, 6,916 val, 19,008 test**.

## 7.2 Test metrics

| Metric  | Value  | Interpretation                                             |
|---------|--------|------------------------------------------------------------|
| HR@1    | 0.1874 | true next POI ranked #1 on 18.7% of test cases             |
| HR@5    | 0.4760 | true next POI in top 5 on 47.6% of test cases              |
| HR@10   | 0.5879 | true next POI in top 10 on 58.8% of test cases             |
| NDCG@5  | 0.3386 | rank-discounted hit rate at k=5                            |
| NDCG@10 | 0.3751 | rank-discounted hit rate at k=10                           |
| MRR     | 0.3163 | average reciprocal rank $\approx 1/3.16 \approx$ rank 3.16 |

## 7.3 Comparison to literature

Numbers below are HR@1 / Acc@1 on Foursquare NYC reproduced from Table 3 of the LLM4POI paper. Our model is bolded.

| Model                       | HR@1 (NYC)   | Family                                |
|-----------------------------|--------------|---------------------------------------|
| LSTM                        | 0.130        | Pure sequence model                   |
| STGCN                       | 0.180        | Spatio-temporal GCN                   |
| <b>This baseline (ours)</b> | <b>0.187</b> | <b>Hybrid GCN + GRU + user (TLMR)</b> |
| STAN                        | 0.220        | Self-attention spatio-temporal        |
| GETNext                     | 0.240        | Transformer + trajectory flow         |
| STHGCN                      | 0.270        | Hyper-graph spatio-temporal           |
| LLM4POI                     | 0.340        | Frozen LLM with prompt engineering    |

**Reading the result.** HR@1 = 0.187 sits between STGCN (0.180) and STAN (0.220) — i.e. at the very top of the "LSTM/STGCN tier" targeted as an honest baseline. The spec explicitly warns that HR@1 above 0.20 with this architecture is "suspicious — re-check the chronological split and graph construction for leakage." We are below that threshold, validating that the chronological split, the training-only co-visit edges, and the per-user prefix generation are all clean.

# 8. Discussion

## 8.1 What worked

- **The hybrid graph structure was useful.** Most POIs land near the kNN-only floor of 10 neighbors, but high-traffic POIs accumulate up to 79 neighbors via the co-visit edges. The fact that no POI was



isolated means the GCN never silently fails to propagate.

- **Per-step concat (not gating).** Concatenating context with the GCN POI feature is the simplest possible fusion and worked well at this scale. Path B in the project roadmap proposes moving  $\Delta_d, \Delta_t$  into ST-GRU gates — that is the next obvious experiment but was deliberately not done here.
- **User embedding contributed.** Foursquare users are a stable population; per-user preferences over POIs are strong enough that a 64-dim lookup adds measurable signal on top of the session prefix.

## 8.2 What we are not claiming

- We are **not** claiming SOTA. The baseline lands  $\approx 8$  HR@1 points below GETNext and  $\approx 15$  below LLM4POI. Both rely on considerably more sophisticated mechanisms (attention over trajectory flow graphs, frozen LLM in-context reasoning) that are out of scope here.
- We are **not** claiming the architecture is novel. The TLMR three-stage shape (graph features + sequence + user) is standard in the POI literature; the value of this work is a clean, reproducible reference implementation honest about its tier.

## 8.3 Honest caveats

- **Single seed.** Results are reported for seed 42 only. POI-rec results in the literature have known sensitivity to initialization; reporting mean  $\pm$  std across 3+ seeds is planned for the thesis (Week 9 of the roadmap).
- **NYC only in this report.** The original spec also covers TKY; that run is intentionally deferred. The pipeline is dataset-agnostic so adding TKY is a one-cell change.
- **No ablations yet.** The next planned experiment removes each component (GCN, user embedding, context) one at a time to quantify its contribution. Until that is done we cannot say *which* piece is doing the work — only that the assembly lands in the expected band.

## 8.4 Path B — beyond the baseline

Four high-leverage extensions are listed in the project roadmap, ordered by expected impact:

- Replace the MLP-to- $|V|$  head with inner-product scoring  $\hat{y}_v = (h_T + e_u)^T h_v^{GCN}$ . Same TLMR shape, much better transfer of graph signal.
- Move  $\Delta_d, \Delta_t$  into ST-GRU gates (STGN-style) instead of side concat. Typically gains 3–5 HR points.
- Add hour-of-day and day-of-week cyclic features at every step. Typically gains 2–4 HR points on Foursquare.
- Weight the co-visit edges by transition frequency (DLIR-style adaptive adjacency) instead of unweighted.

# 9. Phase 2 — itinerary recommendation

A second phase extends the next-POI baseline to **itinerary recommendation**: given a query (start POI, fixed end POI, and a length budget  $K$ ), produce an **ordered route** of  $K$  POIs. Quality is measured by **pairs-F1** (Chen 2016) — F1 over correctly-ordered POI pairs — on the length $\geq 3$  test subset ( $n=2,880$ ), where ordering actually matters (length-2 sessions are trivially perfect).

## 9.1 Two strategies

- **Strategy A — frozen rollout.** Decode the *frozen* next-POI model as an itinerary generator: iterate it from the start, mask visited POIs (loop-free), reserve the fixed end for the last hop. No retraining.
- **Strategy B — pointer network (trained).** Reuse the GCN + user encoder; replace the MLP head with an inner-product pointer; train end-to-end on whole trajectories (teacher forcing), early-stop on val pairs-F1. B-v1 omits  $\Delta d/\Delta t$  context; **B-v2** adds it back into the decoder input.

## 9.2 Results (NYC, length $\geq 3$ test, n=2,880)

| Method                      | pairs-F1       | set-F1 | exact | vs floor  |
|-----------------------------|----------------|--------|-------|-----------|
| A — frozen rollout (greedy) | 0.2887         | 0.609  | 0.054 | — (floor) |
| A — frozen rollout (beam 3) | 0.2902         | 0.610  | 0.057 | +0.0015   |
| B-v1 — pointer, no context  | 0.2585         | 0.578  | 0.043 | -0.0302   |
| B-v2 — pointer + context    | (run on Colab) | —      | —     | —         |

**Reading the result.** Beam barely beats greedy in Strategy A (+0.5%) — the next-POI scorer is myopic, so better *decoding* cannot fix it. More importantly, the from-scratch trained pointer (B-v1) **under-performs the frozen baseline by 0.030 pairs-F1 (-10.5%)**. This is a genuine negative result (B-v1 trained cleanly: loss 8.01 $\rightarrow$ 3.41, val pairs-F1 peaking at 0.271).

**Why the frozen model wins.** Its next-POI engine was trained on  $\sim 75,000$  prefix $\rightarrow$ next examples (every prefix, all lengths), whereas B-v1 saw only 10,281 whole-trajectory examples and used leaner per-step inputs (no  $\Delta d/\Delta t$  context). The honest lesson: *naively training a dedicated itinerary model does not automatically beat cleverly decoding a strong next-POI model*. B-v2 (context-aware pointer) is the implemented next test of whether the missing  $\Delta d/\Delta t$  features close the gap.

## Appendix A — Implementation summary

The codebase is split into a thin `src/` package and a Colab-ready notebook (`train_poi.ipynb`) that imports from it. Module map:

| Module                                | Public API                                                                                                                                                                       |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/data/preprocess.py</code>   | <code>load_foursquare</code> , <code>iterative_filter</code> , <code>reindex</code> , <code>haversine_km</code> , <code>build_sessions</code> , <code>chronological_split</code> |
| <code>src/data/graph.py</code>        | <code>build_covisit_edges</code> , <code>build_knn_edges</code> , <code>build_hybrid_graph</code>                                                                                |
| <code>src/data/dataset.py</code>      | <code>POISessionDataset</code> , <code>collate_fn</code>                                                                                                                         |
| <code>src/models/components.py</code> | <code>POIGraphEncoder</code> , <code>ContextEncoder</code>                                                                                                                       |
| <code>src/models/next_poi.py</code>   | <code>NextPOIModel</code>                                                                                                                                                        |
| <code>src/train.py</code>             | <code>make_loaders</code> , <code>train_one_epoch</code> , <code>train_model</code>                                                                                              |
| <code>src/eval.py</code>              | <code>evaluate_model</code> , <code>fmt_metrics</code>                                                                                                                           |

Tests: 24 pytest cases including a Section-6 smoke test, a controlled-rank evaluation test, padding/dtype tests, and shape tests for every module — all pass on a fresh CPU-only environment.

## Appendix B — Software stack

| Component         | Version                  |
|-------------------|--------------------------|
| Python            | 3.10+ (Colab default)    |
| PyTorch           | 2.3.0                    |
| PyTorch Geometric | 2.5.3                    |
| NumPy             | 1.26.4                   |
| pandas            | 2.1.3                    |
| scikit-learn      | 1.4.2                    |
| pyarrow           | 15.0.2 (parquet I/O)     |
| tqdm              | 4.66.4                   |
| pytest            | 8.1.1 (test runner only) |
| Hardware          | Google Colab T4 (16 GB)  |

## Appendix C — Reproducibility checklist

- Single global seed (42) propagated to random, numpy, torch, and CUDA RNG.
- All preprocessed artifacts (parquets, edge\_index, meta.json) persisted to Google Drive at /MyDrive/poi-rec/data/processed/NYC/ so the training cell can be re-run without redoing preprocessing.
- Best-by-val-HR@10 checkpoint and per-epoch metric history written to disk after every epoch — sessions can be resumed from checkpoints/NYC/latest.pt after a Colab disconnect.
- Public repository: [github.com/6ym6n/PFE\\_IMPLEMENTATION](https://github.com/6ym6n/PFE_IMPLEMENTATION) with pinned requirements.txt, pytest suite, and this report.